

Adopting Code Reviews for Agile Software Development

Mario Bernhart, Andreas Mauczka, Thomas Grechenig

Research Group for Industrial Software

Vienna University of Technology

Vienna, Austria

{mario.bernhart, andreas.mauczka, thomas.grechenig}@inso.tuwien.ac.at

Abstract—Code reviews have many benefits, most importantly to find bugs early in the development phase and to enforce coding standards. Still, it is widely accepted that formal code reviews are time-consuming and the practical applicability in agile development is controversial. This work presents a continuous differential-based method and tool for code reviews. By using a continuous approach to code reviews, the review overhead can be reduced and the effectiveness and applicability in agile environments shall be improved.

Keywords—Tools; Code design; Code tuning; Code reviews; Continuous;

I. INTRODUCTION

Code reviews have many benefits [1, 2, 3], most importantly to find bugs early in the development phase and to enforce coding standards. Still, it is commonly accepted that formal code reviews are time-consuming and expensive [4]. Due to the lack of time and resources [5] many agile software development teams do not perform formal code reviews as a regular task during their development process. Especially at the end of one sprint completing a development task is usually of higher priority than reviewing. Further, even though the tool support for code reviews has made significant progress [6] and supports distributed development [7], code reviews still add a significant overhead in the development process [4].

The research described in this work aims towards a continuous inspection method and tool for agile software development. Continuous in this context means that code inspections are differential-based and automatically triggered by change-sets. The proposed method shall reduce the review overhead and improve the developer acceptance of code inspections. A traditional and formal code review as in [1, 2] often implies that the code under review is one finished artifact. As an alternative approach only the changes of one revision to one other revision of one artifact are reviewed.

The review method was implemented and presented to a large audience (see [8]) and has evolved into a widely used review tool¹. The broad acceptance indicates a significant improvement over traditional review methods. Still, the result has to be evaluated by future research.

II. RELATED WORK

Remillard in [6] discusses some tools that support differential-reviewing, but without any support of automatic scheduling of the reviews e.g. based on changesets. [11] proposes a change-based algorithm to calculate the

accountability of developers to one development artifact and to assess the quality in order to indicate the need for a code review. Furthermore, [11] points out that code reviews have a social effect on the team and should be anonymous to a certain degree. In [3] Baker describes a change-based approach where code-reviews are scheduled automatically as weekly tasks from the changes in the source code management system (SCM). [7, 9] discuss the importance of code reviews for distributed teams. Chen et al. in [10] advocate that collaboration between developers should be rendered within the IDE and [26] discusses the use of groupware support for software inspections.

III. CODE REVIEWS AND AGILE SOFTWARE DEVELOPMENT

Code reviews within agile software development are controversial. Since pair programming inherently employs a certain kind of peer-review, this may be conflicting. Winkler and Biffl [25] compare pair-programming and a best-practice code inspection method and conclude that a mix of inspection and pair programming can be expected to work better than any single technique. Formal inspection methods such as in [2] tend to be too heavyweight for a practical use in agile processes. However, there is a growing interest for more lightweight and tool-supported code reviews in the agile community [24]. Further, it can be assumed that code reviews fit well with major agile techniques like collective code ownership and refactoring.

IV. CODE REVIEW PROCESS

The proposed method of this research mainly relates to the work of Baker as in [3]. Code reviews are scheduled automatically. Compared to [3] this is done concurrently after the SCM commit and not only once a week. Further, in [3] only one reviewer was responsible to review the work of 3-6 developers. In contrast to that, the proposed method uses a flexible assignment of authors and reviewers e.g. to support developer peer reviews in the form {<author, reviewer, filter>,*}. Example mappings are <john, mike, all changes> or <all authors, chris, all testcases>. The proposed method is based on the following workflow:

1. The developer commits to the SCM
2. If there is a mapping that matches the author and the changed items, the review is automatically created for the respective reviewers
3. The review is executed by using a differential-view to the previous version of each file in the change-set and the results are documented

¹ <http://www.inso.tuwien.ac.at/projects/reviewclipse/>

4. The review result is consumed by the author
5. A (negative) review result is followed up and may create a corrective task for the author

A. Process Coherence

According to Boehm [14] and Mizuno [20] the time between the error injection and error identification should be minimized. Williams and Cockburn [23] state that agile methods are mainly based on feedback. With an immediate scheduling approach, the reviews may be executed directly after the commit of the code. This also minimizes the time for the author to get a review result. With a shorter turnaround time from commit to the review result, the correction can be more time-efficient since the developer maybe still working on the same artifact.

In a typical agile continuous integration environment, the developers are committing in short cycles. This would lead to review effort that is evenly distributed within one sprint and therefore improves the chances that there are time and resources to perform the review. Compared to this, in a traditional environment code reviews are typically scheduled at the end of one iteration and therefore create a significant and often challenged schedule factor.

B. Information Coherence

Zimmermann et al. [15] state that all changes that are made within one timeframe have strong information coherence. Traditional review methods use structural information such as modules or components to select the code under review. From a process perspective a set of correlating changes may be distributed over many structural items (e.g. a module and a corresponding unit test). For the proposed review method, all changes of one changeset define the set of code under review. With this approach all changes - e.g. in the code and the corresponding changes in testcases - are part of one review task.

One other important aspect is the size of the code under review. Some studies [16, 17] investigated the ideal size for code reviews. It is commonly accepted that a smaller set of code leads to a more effective and efficient code review. Gilb et al. [18] propose a maximum of 200 lines of code or 2 hours per review. In general, the changeset-based approach reduces the size of one single review task compared to a traditional approach.

Following [10], to support the developers accepting code reviews as a daily task – like refactoring or unit-testing – the reviews shall be tool supported and be rendered directly within the IDE.

V. CODE REVIEW TOOL

Based on the proposed method, the Eclipse-integrated code review tool *Reviewclipse* (RC) was presented [8]. Since the first release early 2009 RC was downloaded 10.000+ times. There is an active and constantly growing user community, giving feedback and requesting features frequently. RC integrates with many popular source code management systems to fetch the changes to automatically schedule the code reviews.

One early Agilo²-integrated prototype that supported the proposed method was not accepted for day-to-day work. The developers reported in [19], that switching tools would prevent them from doing code reviews on a daily basis.

A. Review Configuration

The reviews are scheduled automatically based on a review configuration. The configuration is a set of filtered assignments between an author and a reviewer. The filter selects the files of the changeset for review. Table 1 shows an example assignment where Mike is the senior developer and reviews the Java and XML of Peter, John and Ben. Chris is the test expert and reviews all Java testcases of all team members.

TABLE I. EXAMPLE ASSIGNMENTS FOR REVIEWS

ID	Review Assignments		
	Author	Reviewer	Filter
1	Peter	Mike	*.java, *.xml
2	John	Mike	*.java, *.xml
3	Ben	Mike	*.java, *.xml
4	(* = allAuthors)	Chris	*Test*.java

The review assignments and the review data are stored in XML and distributed via the SCM. Compared to many other source code review systems (see [6]) there is no need for a separate review server. This makes the tool appropriate for small development or student teams. Bernhart et al. in [21] report the successful use of the tool in software engineering education.

B. Review Scheduling and Execution

The reviews are scheduled and executed within a review editor (Figure 1). This editor shows all incoming and outgoing reviews relevant to one user. The main table shows all changesets under review. The lower, left part of the editor shows all changed and filtered files from the selected changeset. A file may be compared with the previous version, or opened for the latest revision. In a typical continuous review the files will be compared to the previous revision.

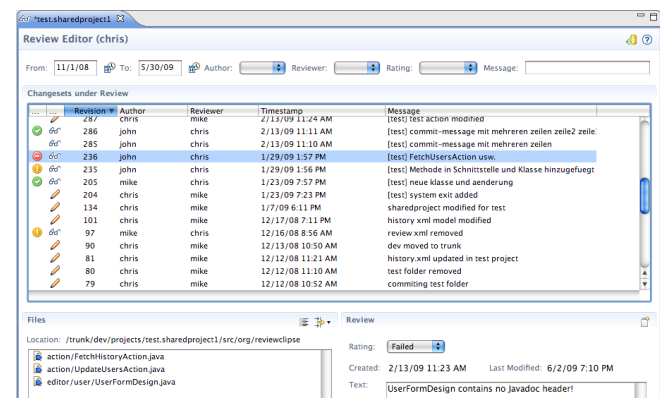


Figure 1. Review Editor

The lower, right part of the editor is used to rate and comment the review. The review is rated with *Passed*, *Warning*

² <http://www.agile42.com/cms/pages/agilo/>

or *Failed*, depending on the severity of any errors found. E.g. a coding error is rated as *Failed* whereas a missing documentation may be rated as a *Warning*.

Figure 2 shows the default compare editor. Typically, each file of the changeset under review will be opened with the compare editor to enable the differential-based reviewing. When the review is done continuously (and therefore includes all consecutive changes) only the differential needs to be reviewed to cover the complete artifact. In the example below 5 lines of deleted code have to be reviewed since all other lines are unchanged and were already reviewed by a previous review.

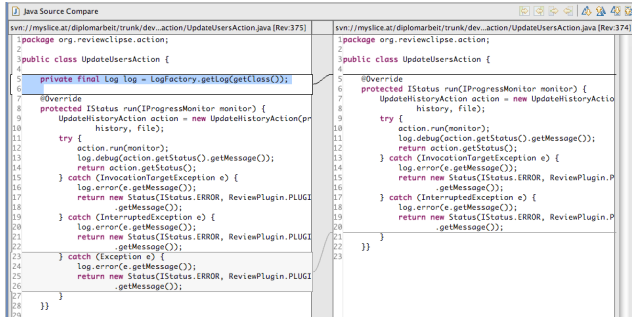


Figure 2. Compare Editor

C. Integration with Issuetracking

Any review result may create a task in an issue-tracking system. To follow up the review result and to schedule the corrective re-work all information about the review is attached to the task. This is implemented as an action in the review editor using adaptors for the most common issue tracking systems.

VI. DISCUSSION

The application of the proposed code review method shows promising early results, but needs proper evaluation. The proposed method provides an alternative to a traditional approach. However, performance of the approach strongly depends on the review scenario. Assumingly, each approach performs better in different review scenarios. It has to be further investigated under which conditions a certain approach has to be preferred.

It is controversial at what point in the continuous integration process a review shall be scheduled. The proposed method is based on post-commit reviews whereas some tools are supporting pre-commit reviews (see Remillard in [6]). The main argument for pre-commit reviews is to find bugs before the code gets submitted to the shared source repository.

Code reviews improve the knowledge sharing within the development team (see Gilb et al. [18]). Since the proposed method is an ongoing process performed in parallel with the development, knowledge sharing does not only cover the stable state of one artifact but also the creation process. It can be assumed that knowledge of the creation process will allow the reviewer to gain a deeper understanding of the authors work and to give a more qualitative review.

Performing code reviews may also affect the code ownership structures. An ongoing knowledge sharing is a basis for collaborative code ownership, which is by itself a major agile quality assuring technique as Nordberg shows in [22].

VII. FUTURE WORK

The future work aims to evaluate the effectiveness and efficiency of the proposed method versus a traditional method of code reviews. The evaluation will be carried out as an empirical study with professional agile teams. The teams will perform both methods in parallel on the same codebase. The effectiveness will be assessed by an expert evaluation combined with a comparison of the review outcome (e.g. found issues and the severities). For the efficiency the total review effort and re-work effort will be measured. Further work includes the investigation of the effect of the proposed method on the knowledge sharing, the code ownership structure and the refactoring.

Based on the statement from Berczuk [12] that for traceability reasons any change should be related to a task e.g. in a task management system, the method will be elaborated to task-based reviews. To achieve a task-based review, the tool will be integrated with Mylyn [13] as a subproject under the name Mylyn Reviews³.

REFERENCES

- [1] M. Fagan, "Design and code inspections to reduce errors in program development," *IBM Systems Journal*, vol. 15, no. 3, pp. 182–211, 1976.
- [2] IEEE standard for software reviews and audits. IEEE STD 1028-2008, pages 1–52, 8 2008.
- [3] R. Baker, "Code reviews enhance software quality," In *Proceedings of the 1997 (19th) International Conference on Software Engineering*, pp. 570–571, May 1997.
- [4] M. Ciolkowski, O. Laitenberger, and S. Biffl. "Software reviews, the state of the practice." *Software*, IEEE, 20(6):46–51, Nov.-Dec. 2003.
- [5] L. Harjumaa, I. Tervonen, and A. Huttunen. "Peer reviews in real life - motivators and demotivators." *Fifth International Conference on Quality Software*, 2005, pages 29–36, Sept. 2005.
- [6] J. Remillard. "Source code review systems." *IEEE Software*, 22(1):74–77, Jan.-Feb. 2005.
- [7] B. Meyer. "Design and code reviews in the age of the internet." *Commun. ACM*, 51(9):66–71, 2008.
- [8] M. Bernhart, C. Mayerhofer, T. Grechenig, "Reviewclipse - Supporting Code-Reviews within the Eclipse IDE". Presentation at EclipseCon09
- [9] M. Stein, J. Riedl, S. J. Harner, and V. Mashayekhi, "A case study of distributed, asynchronous software inspection," in *ICSE '97: Proceedings of the 19th international conference on Software engineering*, (New York, NY, USA), pp. 107–117, ACM, 1997.
- [10] L.-T. Cheng, C. R. de Souza, S. Hupfer, J. Patterson, and S. Ross, "Building collaboration into IDEs," *Queue*, vol. 1, no. 9, pp. 40–50, 2004.
- [11] C. R. Prause and S. Apelt, "An approach for continuous inspection of source code," in *Proceedings of the 6th international workshop on Software quality*, (New York, NY, USA), pp. 17–22, ACM, 2008.
- [12] S. P. Berczuk, "Software configuration management patterns - Effective Teamwork", Practical Integration. Addison-Wesley, 2003.
- [13] Kersten, M. and Murphy, G. C. "Using task context to improve programmer productivity." In *Proceedings of the 14th ACM SIGSOFT international Symposium on Foundations of Software Engineering* (Portland, Oregon, USA, November 05 - 11, 2006). SIGSOFT '06/FSE-14. ACM, New York, NY, 1-11.
- [14] B. W. Boehm, *Software Engineering Economics*. Prentice-Hall, 1981.
- [15] T. Zimmermann, P. Weißgerber, S. Diehl, and A. Zeller. „Mining Version Histories to Guide Software Changes.” Saarland University,

³ <http://www.eclipse.org/reviews/>

- September 2003. *Proc. 26th International Conference on Software Engineering (ICSE)*, Edinburgh, UK, May 2004.
- [16] A. Porter, H. Siy, A. Mockus, and L. Votta, "Understanding the sources of variation in software inspections," *ACM Trans. Softw. Eng. Methodol.*, vol. 7, no. 1, pp. 41–79, 1998.
 - [17] M. Stein, J. Riedl, S. J. Harner, and V. Mashayekhi, "A case study of distributed, asynchronous software inspection," in *ICSE '97: Proceedings of the 19th international conference on Software engineering*, (New York, NY, USA), pp. 107–117, ACM, 1997.
 - [18] T. Gilb, D. Graham, and S. Finzi. "Software Inspection." Addison-Wesley Longman Publishing Co., Inc. Boston, MA, USA, 1993.
 - [19] C. Mayerhofer. "Design and Development of a Module for Change-Set Based Code Reviews in Integrated Development Environments" Master Thesis, Vienna University of Technology, 2009.
 - [20] Y. Mizuno. "Software quality improvement," *Computer*, vol. 16, pp. 66–72, März 1983.
 - [21] M. Bernhart, C. Mayerhofer, T. Grechenig. „Reviewclipse@Class - Using a Lightweight Continuous Software Reviewing Tool in Undergraduate Software Development Education“ – to appear in *Proceedings of the 23rd Annual IEEE-CS Conference on Software Engineering Education and Training*
 - [22] M. E. Nordberg III, "Managing Code Ownership," *IEEE Software*, pp. 26–33, March/April, 2003
 - [23] Williams, L. and Cockburn, A. Agile Software Development: It's All about Feedback and Change, *IEEE Computer*, Vol. 36, No., 6, pp. 83–85, June 2003.
 - [24] W. Seliga, S. Ginter, "Effective code reviews in agile teams" Talk at Agile 2009
 - [25] D. Winkler and S. Biffl. "An empirical study on design quality improvement from best-practice inspection and pair programming." *Product-Focused Software Process Improvement*, pages 319–333, 2006.
 - [26] P. Grünbacher, M. Halling, and S. Biffl. "An empirical study on groupware support for software inspection meetings." *Automated Software Engineering*, International Conference on, 0:4, 2003.